Recursion

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University), Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Aside

I asked the question, why are the return types not considered when looking at function prototypes.

- To clarify, in Java, we can have something like the following:
  - int myfunc(int a, int b, double c)
  - int myfunc(int a, string b, double c)
- The question was, why can't you also have something like this:
  - int myfunc(int a, int b, double c)
  - String myfunc(int a, int b, double c)

Recursion

With F1WAE, we had recursion "for free" because

- All functions were globally scoped (they were in deffun form and stored in a list)
  - So a function was in scope in its own body

The situation is not so simple in the case where functions are not globally scoped, such as with FAE.

Recursion

So, what do we need if we want to have recursion?

- A function that calls itself must refer to itself *by name* inside its own body. So that name has to be **bound to something that is visible inside the body**.

In F(W)AE, the only binding forms are function parameters and **`with`** (which is just sugar for a function application). Let's try both.

Recursion

- A function that calls itself must refer to itself *by name* inside its own body. So that name has to be **bound to something that is visible inside the body**.
- Let's suppose we want to do this with a **`with`** statement. And suppose we want to have recursion for a general function **`f`**
  - We want **`f`** to be bound to something that captures the behavior of **`f`**

```
{with {f {lam {x} {f x}}}
  {f 0}}
```

Recursion

- This is the simplest **`with`** example we can have
  - The lambda captures the "behavior" of **`f`**
  - This behavior is bound to **`f`** by the **`with`**
  - Then in the body of the **`with`**, we call **`f`**
  - What happens when we do?

```
{with {f {lam {x} {f x}}}
  {f 0}}
```

Recursion

- This is the simplest **`with`** example we can have
  - Then in the body of the **`with`**, we call **`f`**
  - What happens when we do?

```
{with {f {lam {x} {f x}}}
  {f 0}}
```

  - The **`f`** in **`{f 0}`** is bound to the binding occurrence of **`f`**. When we try to execute that **`f`** with 0 input, the **`f`** in the body of the lambda causes a free variable error – the lam evaluates to a closure that captures the environment in which the lambda is created: (x, {f x}, empty). In that environment, **`f`** is free!

Recursion

- What is the function call example we can have?
- Well, how do we transform a with to an equivalent function call?

In general,

{with {<id> < F1WAE >$_1$} < F1WAE >$_2$}

Is the same as:

{{lam {<id> } < F1WAE >$_2$} < F1WAE >$_1$}

```
{with {f {lam {x} {f x}}}
  {f 0}}
```

Recursion

- What is the function call example we can have?
- Well, how do we transform a with to an equivalent function call?

In general,

{with {<id> < F1WAE >$_1$} < F1WAE >$_2$}

Is the same as:

{{lam {<id> } < F1WAE >$_2$} < F1WAE >$_1$}

```
{with {f {lam {x} {f x}}}
  {f 0}}
```

```
{{lam {f} {f 0}}
 {lam {x} {f x}}}
```

Recursion

- Bottom line: neither with nor function application does what we want.

```
{with {f {lam {x} {f x}}}
  {f 0}}
```

```
{{lam {f} {f 0}}
 {lam {x} {f x}}}
```

- So we have to get creative to in order to get recursion in the FAE context

The Roadmap: Why JavaScript?

- We'll follow the article Y: The Most Beautiful Idea in Computer Science, which uses JavaScript, then translate the result to Racket and FAE
- Why JavaScript?
  - Its syntax is close to Java's, so hopefully more natural for you
  - It has a conditional and multiplication, so we can use factorial
  - It is untyped. PLAIT's type checker rejects the key step: applying a function to itself
  - Its functions translate directly into FAE's lam and application

The article can be found here:
https://www.lucasfcosta.com/blog/Y-The-Most-Beautiful-Idea-in-Computer-Science

A Caveat About JavaScript

- In JavaScript, a function can call itself by name:

```
const factorial = n => n === 0 ? 1 : n * factorial(n - 1);

console.log(factorial(5)); // 120
```

- This works because the body looks up factorial when it runs, and by then the name is bound
- FAE's with does not allow this, so we will follow FAE's rule:
- No function may refer to its own name
- Goal: write factorial under that rule

JavaScript Arrow Functions

```
x => x * 2            // a function of x, like Java's  x -> x * 2
(x, y) => x + y       // two parameters
n === 0 ? 1 : 2       // conditional expression, same as Java

const add = x => y => x + y;    // curried: x => (y => x + y)
add(3)(4);                      // 7
```

- x => e is FAE's {lam {x} e}, and f(x) is FAE's {f x}
- Curried arrows take one argument at a time, just like FAE's lam

Step 1: Recursive Call as a Parameter

- Replace the recursive call with a call to a parameter f:

```
const factorialGenerator = f => n => n === 0 ? 1 : n * f(n - 1);
```

- factorialGenerator is not recursive: it never mentions its own name
- It takes f and returns a function of n
- Which f makes that function compute factorial?
- a function like factorialGenerator, that takes a "recursive call" as a parameter f and returns the function we want is called a generator

Step 2: Try Any Function as f

```
const literallyAnyFunction = () => {
    console.log('f has been called!')
    return 'a string'
}

console.log(factorialGenerator(literallyAnyFunction)(0)); // ?
console.log(factorialGenerator(literallyAnyFunction)(1)); // ?
```

Step 2: Try Any Function as f

```
const literallyAnyFunction = () => {
   console.log('f has been called!')
   return 'a string'
}

console.log(factorialGenerator(literallyAnyFunction)(0)); // 1
console.log(factorialGenerator(literallyAnyFunction)(1)); // NaN
```

- n = 0 works: the base case never calls f
- n = 1 calls f (the message prints), and 1 * 'a string' is NaN

Step 3: Nest factorialGenerator

```
const result = factorialGenerator(
   factorialGenerator(
      factorialGenerator(literallyAnyFunction)
   )
)(2);

console.log(result); // ?
```

Step 3: Nest factorialGenerator

```
const result = factorialGenerator(
    factorialGenerator(
        factorialGenerator(literallyAnyFunction)
    )
)(2);

console.log(result); // 2
```

- With two nested factorialGenerators, input 1 works but 2 gives NaN. With three, 2 works
- Each extra factorialGenerator handles one more level of recursion
- Factorial of n needs n + 1 of them

To Handle Every n

```
const factorial = factorialGenerator(
   factorialGenerator(
      factorialGenerator(
         factorialGenerator(...)
      )
   )
);
```

- We would need infinitely many nested factorialGenerators
- We can't write that out, but there's a precise way to describe it: a fixed point

Fixed Points

- x is a fixed point of a function g if g(x) = x
- Example: cos(x) has a fixed point near 0.739
  - Press cos repeatedly on a calculator and you converge to it

```
Math.cos(0.7390851332151607);  // 0.7390851332151607
```

- What is a fixed point of factorialGenerator?

Factorial Is a Fixed Point

- Substitute factorial for f in factorialGenerator:

```
factorialGenerator(factorial)
  = n => n === 0 ? 1 : n * factorial(n - 1)    // what we get

factorial
  = n => n === 0 ? 1 : n * factorial(n - 1)    // its definition
```

- They are the same function:
- factorialGenerator(factorial) = factorial
- So factorial is a fixed point of factorialGenerator

Note: we are not allowed to use factorial in our solution

Why That Helps

- Our problem becomes: find the fixed point of factorialGenerator, without self-reference
- A general fixed-point finder Y would do this for any function:
  - Ex. Y(factorialGenerator) would be factorial
- Check: unrolling the fixed-point equation gives our infinite nesting

```
factorial = factorialGenerator(factorial)
          = factorialGenerator(factorialGenerator(factorial))
          = factorialGenerator(factorialGenerator(...))
```

This reframes the whole problem. Instead of writing factorial with self-reference, we look for the fixed point of a non-recursive function. If we can write one function Y that finds fixed points, every recursive function becomes Y applied to its generator.

The Goal: A Fixed-Point Finder

- We want a function Y with Y(f) = f(Y(f)) for any generator f
- We've seen that then Y(factorialGenerator) is factorial. The same works for any recursive function
- Our rule still applies: Y may not refer to its own name
- A function with no free variables is called a combinator. That's what we need Y to be

The Y Combinator

```
const yCombinator = f => (g => g(g))(g => f((x) => g(g)(x)));

const factorial = yCombinator(factorialGenerator);
console.log(factorial(5)); // 120
```

- No names and no free variables, so it is a combinator
- This is the article's final line
- Strictly, it's the Z combinator: Y for eager languages

Parsing the Y Combinator

```
const yCombinator = f => BODY

BODY = (g => g(g))  (g => f((x) => g(g)(x)))
       └── A ──┘    └──────────── B ──────────┘
```

- BODY is one function application: A applied to B
- A = g => g(g) takes a function and applies it to itself
- B = g => f((x) => g(g)(x)) takes g, then calls the generator f on a delayed g(g)
- The two g's are different parameters, each local to its own function

```
const yCombinator = f => (g => g(g))(g => f((x) => g(g)(x)));
```

Evaluating the Y Combinator

```
yCombinator(f)
  = A(B)              // the body: A applied to B
  = B(B)              // A applies B to itself
  = f((x) => B(B)(x))   // B's body, with g = B
```

- (x) => B(B)(x) behaves like B(B), effectively we have f(B(B)) in the last step
- But the second step is B(B). So since the second step is equal to the third step, we have B(B) = f(B(B))
- so R = B(B) satisfies R = f(R): a fixed point of f
- That is, yCombinator(f) = R (by line 2), which is a fixed point of f

Tracing the Y Combinator

- Computing yCombinator(factorialGenerator)(2). With f = factorialGenerator, R = B(B) is:

```
R = n => n === 0 ? 1 : n * B(B)(n - 1)

R(2) = 2 * B(B)(1)        // B(B) builds R again
     = 2 * 1 * B(B)(0)    // and again
     = 2 * 1 * 1          // base case: B(B) is not called
     = 2
```

- Each recursive call builds the function again with B(B), only when the recursion reaches it

Idea 1: Self-Application

```
const factorialRecursive = f => n => n === 0 ? 1 : n * f(f)(n - 1);

console.log(factorialRecursive(factorialRecursive)(5)); // 120
```

- A function can't refer to itself by name, but it can receive itself as an argument
- f(f) rebuilds the function for the next recursive call
- In yCombinator, g(g) plays this role, and `g => g(g)` starts it off without needing a name

Idea 2: The Delay

- Without the delay, the definition is simpler, but it never returns:

```
const noDelay = f => (g => g(g))(g => f(g(g)));

noDelay(factorialGenerator);
// RangeError: Maximum call stack size exceeded
```

- JavaScript evaluates g(g) before calling f, and g(g) evaluates g(g) again, forever
- Wrapping it as (x) => g(g)(x) delays g(g) until the recursive call actually happens
- noDelay is the classic Y combinator, which works in lazy languages

Aside: Who on Earth developed this?

- Multiple folks. And they weren't looking for this
  - Haskell Curry
  - Alonzo Church
  - Alan Turing
  - Stephen Kleene
- They did this before there were computers
- They were studying logic and looking in particular for logical paradoxes.

Fibonacci

```
const fibonacciGenerator = f => n => n <= 1 ? 1 : f(n - 1) + f(n - 2);

const fibonacci = yCombinator(fibonacciGenerator)

console.log(fibonacci(4)); // 5
console.log(fibonacci(5)); // 8
```

- Only the generator changes. yCombinator stays the same
- Write the function as if it could call itself, but take the recursive call as the parameter f

Back to Racket: The Rules

JavaScript	Racket
`x => e`	`(λ (x) e)`
`f(x)`	`(f x)`
`f(x)(y)`	`((f x) y)`
`c ? a : b`	`(if c a b)`
`n === 0`	`(= n 0)`
`n * f(n - 1)`	`(* n (f (- n 1)))`
`const name = e;`	`(define name e)`

- Racket puts the function inside the parentheses: `(f x)`
- Every operator is a function call too: `n * m` becomes `(* n m)`

Back to Racket: yCombinator

- Translate the pieces from Parsing the Y Combinator:

	JavaScript	Racket
A	g => g(g)	(λ (g) (g g))
B	g => f((x) => g(g)(x))	(λ (g) (f (λ (x) ((g g) x))))
BODY	A(B)	(A B)
yCombinator	f => BODY	(λ (f) BODY)

- Inside B: (x) => g(g)(x) becomes (λ (x) ((g g) x)). First (g g), then apply the result to x
- Substituting A and B into BODY gives the Racket y-combinator on the next slide

Back to Racket

```
(define y-combinator
 (λ (f)
  ((λ (g) (g g))
   (λ (g) (f (λ (x) ((g g) x)))))))

(define factorial-generator
 (λ (f)
  (λ (n) (if (= n 0) 1 (* n (f (- n 1)))))))

((y-combinator factorial-generator) 5)   ; => 120
```

- x => e becomes (λ (x) e), and f(x) becomes (f x)
- Use #lang racket: PLAIT's type checker rejects (g g)

And in FAE

```
{lam {f}
  {{lam {g} {g g}}
   {lam {g} {f {lam {x} {{g g} x}}}}}}
```

- Only lam and application, with no free identifiers, so it satisfies FAE's scoping rule
- FAE has no conditional, so we can't run factorial in it yet. Adding one (such as if0) is all it would take
- This answers Lecture 9's question: yes, we can write recursive functions in FAE

Exercise: Applying yCombinator

- Use yCombinator to write:
  - A function that computes the sum of the numbers from 0 to n, given n
  - A function that computes the sum of the numbers in an array, given an array
- Write the generator first. Then translate your solutions to Racket